



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №6
на тему «ШАБЛОНИ «Abstract Factory», «Factory Method»,
«Memento», «Observer», «Decorator»..»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	9
Реалізація шаблону за обраною темою	10
1. Observer.....	10
2. FileSubject	11
3. SyntaxHighlighter	12
4. ChangeLogger	12
5. ObserverConfig.....	13
Посилання на код на GitHub	13
Діаграма класів для патерну «Observer»	14
Переваги застосування патерну «Observer».....	14

ШАБЛОНИ «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Мета: ознайомитися з шаблонами проектування «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Принципи SOLID, запропоновані Робертом Мартіном, визначають правила розробки класів у системах.

- Кожен клас має виконувати лише одну чітко визначену задачу.
Приклад: Якщо служба звітів інтегрована зі службою друку в одному класі, зміни в одній службі можуть порушити роботу іншої.
Відокремлення обов'язків знижує зв'язаність і підвищує цілісність системи.
- Класи мають бути відкритими для розширення, але закритими для зміни.
Реалізація: Використання інтерфейсів або абстрактних класів дозволяє змінювати поведінку без модифікації вихідного коду, наприклад, для додавання нових функцій чи тестування.
- Підкласи повинні зберігати поведінку батьківських класів.
Приклад: Якщо в дочірньому класі метод "сума" реалізований через множення, це порушує принцип. Програма має працювати коректно як із батьківським, так і з дочірнім класом.
- Багато вузьких інтерфейсів краще, ніж один універсальний.

Суть: Кожен інтерфейс має відповідати за конкретну функцію. Це спрощує підтримку й використання системи, дозволяючи взаємодіяти лише з необхідними частинами.

- Залежності повинні будуватися на основі абстракцій, а не деталей.

Реалізація: Верхньорівневі модулі посилаються на інтерфейси, які реалізуються нижньорівневими модулями. Це зменшує зв'язаність і спрощує модифікацію системи.

SOLID-принципи допомагають створювати гнучкі, підтримувані та масштабовані програмні системи.

Шаблон «Abstract Factory»

Призначення: Створює сімейства пов'язаних об'єктів без вказівки їх конкретних класів. Застосовується, коли необхідно забезпечити сумісність продуктів одного сімейства.

Структура:

- загальний інтерфейс для створення продуктів.
- реалізації для різних сімейств продуктів.
- загальний інтерфейс для кожного продукту.
- конкретні реалізації продуктів.
- працює з фабриками та продуктами через загальні інтерфейси.

Проблема: Наприклад, потрібно створювати меблі (крісла, дивани, столики) у різних стилях (Ар-деко, Вікторіанський, Модерн), які пасуватимуть одне до одного.

Рішення:

- Виділяються загальні інтерфейси для кожного продукту.
- Для кожного сімейства продуктів створюються свої фабрики, що реалізують інтерфейс фабрики.
- Клієнт працює лише із загальними інтерфейсами, що забезпечує гнучкість і сумісність.

Приклад: Уявіть симулятор меблевого магазину, який створює меблі в заданих стилях, щоб уникнути їх несумісності.

Переваги:

- Забезпечує сумісність продуктів.
- Звільняє код від залежності від конкретних класів.
- Реалізує принцип відкритості/закритості.

Недоліки:

- Складність через велику кількість класів.
- Обов'язкова наявність усіх типів продуктів у кожній варіації.

Шаблон «Factory Method»

Призначення: Визначає інтерфейс для створення об'єктів певного базового типу, дозволяючи підкласи замінювати об'єкти на їх підтипи. Це спрощує підтримку коду та розширення функціоналу без зміни базового коду.

Структура:

- **Creator:** базовий клас із фабричним методом.
- **ConcreteCreator:** підкласи, що реалізують фабричний метод для створення конкретних об'єктів.
- **Product:** базовий інтерфейс для об'єктів.
- **ConcreteProduct:** реалізації продуктів.

Проблема: Програма для вантажних перевезень спочатку працює лише з вантажівками. Для додавання інших видів транспорту (судна, літаки) потрібно змінювати весь код, що ускладнює його підтримку.

Рішення:

- Замінити пряме створення об'єктів (new) викликом фабричного методу.
- Кожен вид транспорту (Вантажівка, Судно) реалізує спільний інтерфейс Транспорт, а підкласи логістики (Дорожня, Морська) повертають відповідні об'єкти через фабричний метод.

Приклад: Менеджер із найму делегує процес співбесіди різним спеціалістам залежно від вакансії, зберігаючи спільну логіку найму.

Переваги:

- Усунення прив'язки до конкретних класів продуктів.
- Спрощення підтримки коду.
- Легке додавання нових продуктів.

Недоліки:

- Може створити великі паралельні ієрархії класів.

Шаблон «Memento»

Призначення: Шаблон дозволяє зберігати та відновлювати стан об'єкта без порушення інкапсуляції. Об'єкт-знімок (Memento) використовується виключно для збереження стану. Початковий об'єкт (Originator) може створювати та відновлювати свій стан із знімка, тоді як зовнішні об'єкти, такі як Caretaker,

лише зберігають або передають знімки.

Проблема: Наприклад, у текстовому редакторі потрібно реалізувати функцію скасування дій. Клас редактора повинен зберігати свій стан перед кожною зміною. Прямий доступ до полів об'єкта порушує інкапсуляцію та створює труднощі при зміні структури класу.

Рішення: Шаблон «Memento» передбачає, що об'єкт сам створює знімки свого стану й надає обмежений доступ до них іншим об'єктам. Caretaker зберігає ці знімки та передає їх назад для відновлення стану.

Приклад з життя: Клас історії у текстовому редакторі може зберігати список знімків із назвами та датами. Коли користувач виконує скасування, історія передає останній знімок редактору для відновлення.

Переваги:

- Не порушує інкапсуляцію.
- Спрощує вихідний об'єкт, прибираючи необхідність зберігати історію станів.

Недоліки:

- Вимагає багато пам'яті при частому створенні знімків.
- Може перевантажувати пам'ять, якщо застарілі знімки не видаляються.

Шаблон «Observer»

Призначення: Шаблон визначає залежність "один-до-багатьох", дозволяючи об'єктам-спостерігачам автоматично отримувати сповіщення про зміни стану об'єкта-видавця.

Проблема: Постійне ручне перевіряння змін (як покупець, що щодня перевіряє товар у магазині) неефективне, а масова розсилка повідомлень може викликати незадоволення у користувачів.

Рішення:

- Видавець зберігає список підписників.
- Підписники додають або видаляють себе зі списку за допомогою методів видавця.
- При зміні стану видавець повідомляє підписників, викликаючи їх методи.
- Загальний інтерфейс дозволяє підписникам працювати з різними видавцями.

Приклад: Підписка на журнал: видавництво надсилає нові номери без

додаткових запитів із вашого боку.

Переваги:

- Динамічна підписка/відписка.
- Слабке зв'язування між видавцем і підписниками.
- Відповідає принципу відкритості/закритості.

Недоліки:

- Невизначена послідовність сповіщення підписників.

Шаблон «Decorator»

Призначення: Динамічно додає функціональність об'єкту під час виконання програми, зберігаючи його базову поведінку. Декоратор обертає об'єкт, додаючи додаткові функції, що забезпечує більшу гнучкість, ніж спадкування.

Приклад:

Декоратор для бібліотечних елементів, що додає можливість оренди:

```
class Borrowable : Decorator {
    protected List<string> borrowers = new List<string>();
    public Borrowable(LibraryItem libraryItem)
    public void BorrowItem(string name) {
        borrowers.Add(name); libraryItem.NumCopies--;
    }
    public void ReturnItem(string name) {
        borrowers.Remove(name); libraryItem.NumCopies++;
    }
    public override void Display() {
        base.Display();
        foreach (string borrower in borrowers) Console.WriteLine ("Borrower:
        " + borrower);
    }
}
```

Базові функції елементів (наприклад, відображення даних) залишаються незмінними, а оренда додається динамічно.

Проблема:

Уявіть систему сповіщень, яка надсилає повідомлення різними каналами (email, SMS, Slack). Спадкування для кожної комбінації каналів значно роздуває код.

Рішення:

Замінити спадкування композицією. Декоратор обгортає об'єкт, додаючи функції без зміни вихідного класу. Наприклад, Notifier можна обгорнути декораторами для SMS і Slack, зберігаючи email-сповіщення.

Приклад із життя:

Одяг — аналог декоратора. Ви вдягаєте светр для тепла, а плащ — для захисту від дощу, не змінюючи свою сутність.

Переваги:

- Додає функціональність «на льоту».
- Гнучкість у порівнянні зі спадкуванням.
- Відповідає принципу відкритості/закритості.

Недоліки:

- Велика кількість дрібних класів.
- Складність налаштування при багатошаровій обгортці.

Реалізація не менше 3-х класів відповідно до обраної теми:

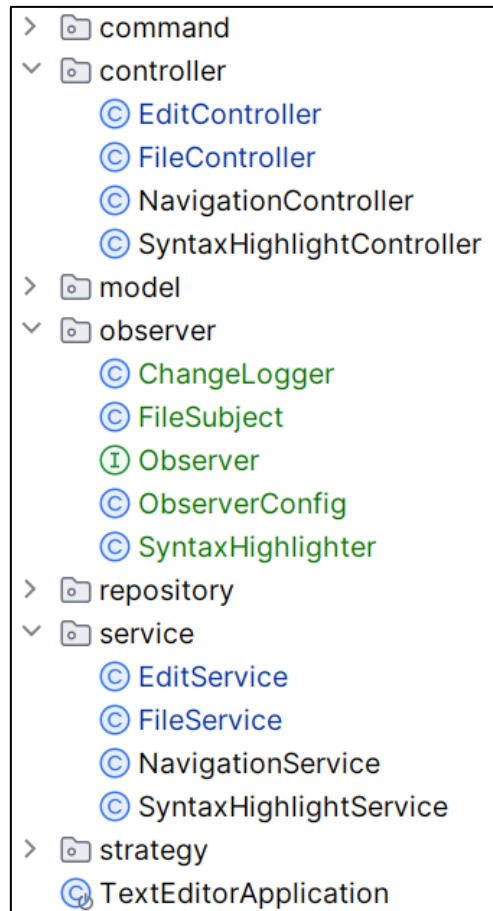


Рисунок 1. Структура проекту з використанням патерну «Observer».

У ході лабораторної роботи була реалізована система текстового редактора, основним завданням якої було використання патерна **Observer** для організації роботи з оновленнями тексту, підсвічуванням синтаксису та логуванням змін. Структура проєкту наведена на Рисунку 1. У результаті виконання роботи були реалізовані наступні кроки::

1. Розроблено структуру проєкту:

- Проєкт поділено на логічні пакети: controller, model, repository, service, strategy, та observer, що забезпечує зручність масштабування та підтримки коду.

2. Реалізовано патерн "Спостерігач" (Observer):

- Створено клас FileSubject, який виступає як суб'єкт (subject) і зберігає список підписників.
- Реалізовано інтерфейс Observer, який визначає метод update(String state) для підписників.
- Додано підписників:
 - SyntaxHighlighter — відповідає за підсвічування синтаксису при зміні тексту.
 - ChangeLogger — веде журнал змін тексту.

- Реалізовано конфігураційний клас ObserverConfig, який автоматично додає підписників до суб'єкта FileSubject.
3. **Реалізовано сервісний рівень:**
- FileService працює з файлами (відкриття, збереження, редагування) і викликає методи setContent() у FileSubject для сповіщення підписників про зміни.
 - EditService обробляє операції з текстом (вставка тексту) і також повідомляє FileSubject про зміни.
4. **Створено контролери для роботи з функціоналом:**
- FileController — забезпечує роботу з файлами (відкриття, збереження, редагування) і викликає оновлення у FileSubject.
 - EditController — відповідає за операції редагування, наприклад, вставка тексту, що також повідомляє FileSubject.
5. **Перевірено функціонал системи:**
- Тестування API підтвердило коректність сповіщення підписників під час змін тексту.
 - Логи показали роботу підписників:
 - SyntaxHighlighter оновлював підсвічування тексту.
 - ChangeLogger записував зміни тексту в лог.

Даний підхід забезпечує автоматичну реакцію підписників на зміни тексту без необхідності прямої взаємодії між ними. Патерн **Observer** дозволяє легко додавати нових підписників або змінювати їх поведінку без змін у основному коді. Це спрощує підтримку та розширення функціоналу текстового редактора, забезпечуючи його гнучкість та масштабованість.

Реалізація шаблону за обраною темою:

1. Observer

1	<code>package com.example.text_editor.observer;</code>
2	
3	<code>6 usages 2 implementations new *</code> <code>public interface Observer {</code>
4	<code>1 usage 2 implementations new *</code> <code>void update(String state);</code>
5	<code>}</code>

Рисунок 2. Інтерфейс Observer.

Цей Інтерфейс Observer, який ми бачимо на Рисунку 1 визначає метод update(String state), який реалізується всіма спостерігачами. Завдяки цьому інтерфейсу можна створити будь-яку кількість спостерігачів для відстеження змін у об'єктах.

2. FileSubject

```
1 package com.example.text_editor.observer;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.stereotype.Component;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10 8 usages new *
11 @Component
12 public class FileSubject {
13     4 usages
14     private static final Logger logger = LoggerFactory.getLogger(FileSubject.class);
15
16     4 usages
17     private final List<Observer> observers = new ArrayList<>();
18     2 usages
19     private String content;
20
21     2 usages new *
22     public void attach(Observer observer) {
23         observers.add(observer);
24         logger.info("Attached observer: {}", observer.getClass().getSimpleName());
25     }
26
27     no usages new *
28     public void detach(Observer observer) {
29         observers.remove(observer);
30         logger.info("Detached observer: {}", observer.getClass().getSimpleName());
31     }
32 }
```

Рисунок 3.1. Клас FileSubject.

```
33 4 usages new *
34 public void setContent(String content) {
35     this.content = content;
36     logger.info("Content updated: {}", content);
37     notifyObservers();
38 }
39
40 1 usage new *
41 private void notifyObservers() {
42     logger.info("Notifying {} observers...", observers.size());
43     for (Observer observer : observers) {
44         observer.update(content);
45     }
46 }
47 }
```

Рисунок 3.2. Клас FileSubject.

Цей клас FileSubject, який ми бачимо на Рисунках 3.1 та 3.2 виконує роль суб'єкта в патерні Observer:

- Має список спостерігачів (observers), які підписуються на зміни.
- Реалізує методи:
 - attach(Observer observer) — додає спостерігача до списку.
 - detach(Observer observer) — видаляє спостерігача зі списку.
 - notifyObservers() — сповіщає всіх спостерігачів про зміну стану.
 - setContent(String content) — встановлює новий стан і викликає сповіщення.

3. SyntaxHighlighter

```
1 package com.example.text_editor.observer;
2
3 import org.springframework.stereotype.Service;
4
5 new *
6 @Service
7 public class SyntaxHighlighter implements Observer {
8     1 usage new *
9     @Override
10     public void update(String state) { System.out.println("Підсвічування синтаксису оновлено для тексту: " + state); }
11 }
```

Рисунок 4. Клас SyntaxHighlighter.

Цей клас SyntaxHighlighter, який ми бачимо на Рисунку 4 є конкретним спостерігачем, який реалізує інтерфейс Observer. Він оновлюється при зміні стану суб'єкта і відповідає за оновлення підсвічування синтаксису, відображаючи повідомлення: "Підсвічування синтаксису оновлено для тексту: {state}".

4. ChangeLogger

```
1 package com.example.text_editor.observer;
2
3 import org.springframework.stereotype.Service;
4
5 new *
6 @Service
7 public class ChangeLogger implements Observer {
8     1 usage new *
9     @Override
10     public void update(String state) { System.out.println("Журнал змін оновлено. Поточний текст: " + state); }
11 }
```

Рисунок 5. Клас ChangeLogger.

Цей клас ChangeLogger, який ми бачимо на Рисунку 5 також є конкретним спостерігачем, який реалізує інтерфейс Observer. Його завдання — оновлювати журнал змін, коли стан суб'єкта змінюється. Лог дій виглядає так: "Журнал змін оновлено. Поточний текст: {state}".

5. ObserverConfig

```
1 package com.example.text_editor.observer;
2
3 import org.springframework.boot.CommandLineRunner;
4 import org.springframework.context.annotation.Bean;
5 import org.springframework.context.annotation.Configuration;
6
7 new *
8 @Configuration
9 public class ObserverConfig {
10
11     new *
12     @Bean
13     public CommandLineRunner registerObservers(FileSubject fileSubject,
14                                             SyntaxHighlighter syntaxHighlighter,
15                                             ChangeLogger changeLogger) {
16
17         return args -> {
18             fileSubject.attach(syntaxHighlighter);
19             fileSubject.attach(changeLogger);
20         };
21     }
22 }
```

Рисунок 6. Клас ObserverConfig.

Цей клас ObserverConfig, який ми бачимо на Рисунку 6 використовується для автоматичної реєстрації спостерігачів у FileSubject під час запуску програми. Він:

- Реєструє спостерігачів (SyntaxHighlighter і ChangeLogger) через метод attach.
- Використовує CommandLineRunner для запуску цього процесу під час старту додатка.

Посилання на код на GitHub: https://github.com/vvvvvvvvvviktoriaa/text_editor

Діаграма класів для патерну «Observer»:

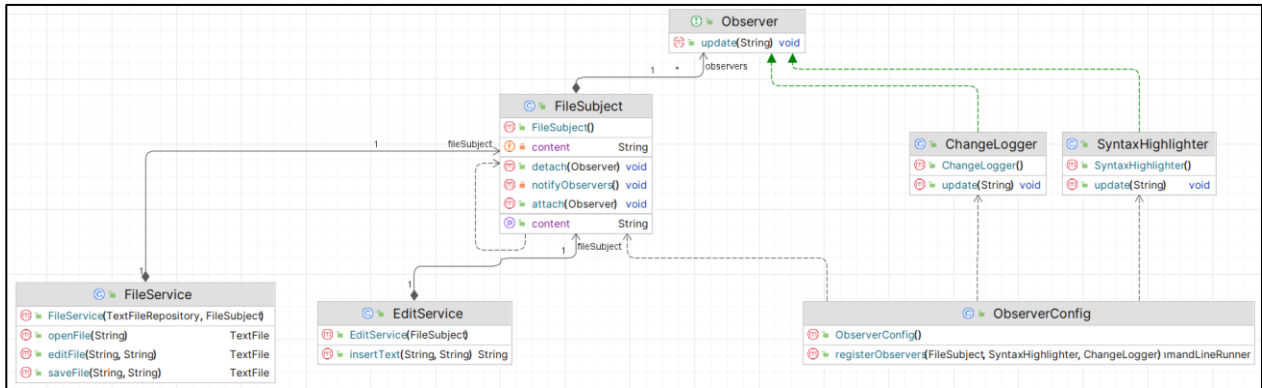


Рисунок 8. Діаграма класів для патерну «Observer».

У ході лабораторної роботи був реалізований патерн «Observer» для текстового редактора, що дозволило організувати ефективну систему сповіщень про зміни вмісту файлів. Завдяки цьому патерну, система забезпечує автоматичне оновлення даних спостерігачів (наприклад, підсвічування синтаксису та журналу змін), коли змінюється вміст файлів.

Раніше для досягнення подібної функціональності, кожний компонент (наприклад, підсвічування синтаксису чи журнал змін) довелося б вручну перевіряти зміни у файлах, що призвело б до:

- Ускладнення підтримки коду через жорстку залежність між компонентами.
- Неможливості ефективно реагувати на зміни в одному об'єкті без додаткових зусиль.
- Порушення принципу відкритості/закритості, оскільки додавання нових функцій вимагало б змін в існуючих компонентах.

Патерн «Observer» вирішив ці проблеми, дозволивши:

- Визначити один об'єкт (суб'єкт), зміни в якому автоматично сповіщають всіх підписаних на нього об'єктів (спостерігачів).
- Ізолювати логіку сповіщень від логіки роботи з іншими компонентами, що дозволяє легко додавати нові спостерігачі без змін у коді суб'єкта.
- Забезпечити більш чисту та гнучку архітектуру, де спостерігачі можуть реагувати на зміни без активного втручання в основну логіку роботи програми.

Переваги застосування патерну «Observer»:

1. Гнучкість:

- Легко додавати нові спостерігачі (наприклад, для додаткових операцій з підсвічуванням або реєстрацією змін) без змін в існуючих класах.

2. Принцип відкритості/закритості:

- Існуючі класи не потребують змін для додавання нових функцій чи спостерігачів.

3. Читаємість і підтримка:

- Логіка сповіщень ізольована, що спрощує тестування, підтримку та розуміння коду.

4. Масштабованість:

- Нові спостерігачі можуть бути додані без зміни в основній логіці програми, що дає змогу легко розширювати функціонал

5. Модульність:

- Компоненти працюють незалежно, що дозволяє їх оновлювати чи замінювати без впливу на інші частини системи.

Висновок: у цій лабораторній роботі був реалізований патерн «Observer» для текстового редактора, що дозволило створити ефективну систему сповіщення про зміни вмісту файлів. Це дозволило організувати гнучкий і модульний механізм для підсвічування синтаксису та журналу змін, що значно спростило підтримку та розширення функціональності редактора.